

Master Thesis Log: Automated Code Smell Prioritization and Ranking

Binamra Aryal

Institution: [Your University]

Advisor: [Advisor Name]

Last Updated: December 13, 2025

Current Phase: Phase 1 (Python Implementation & Smoke Testing)

Contents

Executive Summary	2
1 The Research Problem	3
1.1 The “Noise” Problem in Static Analysis	3
1.2 The Flaw in Existing Ground Truths	3
1.3 The Core Argument: Revealed Preference	3
2 Research Questions & Objectives	4
3 Methodology (The Hybrid Orchestrator)	5
3.1 Overview: The Pipeline-as-Metric-Extractor	5
3.2 Phase 1: Candidate Generation (Static Analysis)	5
3.3 Phase 2: Evidence Gathering (Historical Analysis)	5
3.4 Phase 3: The Fortified Heuristics (Synthesis)	5
3.4.1 Heuristic A: The Weighted Refactoring System	6
3.4.2 Heuristic B: AST-Based Proximity Analysis	6
3.4.3 Heuristic C: Code Criticality (The Survivor Check)	6
4 System Architecture (“Smell-Ranker”)	7
4.1 The Hybrid Cloud Environment	7
4.2 Software Design Patterns	7
4.3 Toolchain Selection: The JDeodorant vs. PMD Decision	7
5 Experimental Design & Data Selection	8
5.1 The “Calibration-First” Strategy	8
5.2 The Selected Calibration Target: <code>refactoring-toy-example</code>	8
5.3 Phase 0 Mining Results (The Verification)	8
5.4 The “Forced-Loop” Validation & Scope Refinement	9
5.4.1 The “Blind Spot” Problem	9
5.4.2 Resolution: The Forced-Loop Strategy	9
5.5 Pipeline Execution Workflow	10
5.6 Metrics Definitions & Justification	10
5.6.1 1. Refactoring Density (ρ)	10
5.6.2 2. Signal Strength (σ)	10
5.6.3 3. Commit Purity (P)	10
5.7 Phase 1 Results: The Calibration Metrics	11
6 Current Project Status	12
Appendix A: Daily Research Log	13

Executive Summary

This research project addresses the critical issue of “alert fatigue” in software maintenance. Static analysis tools (SATs) like SonarQube identify thousands of potential “code smells” (design flaws), but fail to prioritize them based on actual impact. This lack of prioritization forces developers to ignore these warnings, leading to the accumulation of high-risk Technical Debt.

The primary contribution of this research is the development of **Smell-Ranker**, an automated orchestration pipeline that generates a “History-Based Ground Truth.” Unlike existing datasets that rely on subjective expert opinions, Smell-Ranker leverages the economic principle of **Revealed Preference**. It mines software repositories to identify smells that developers have actively spent resources (time/money) to fix (Refactoring) or that have caused demonstrable harm (Bugs).

Currently, the project is in **Phase 1 (Implementation)**. The cloud infrastructure is established, the toolchain (PMD, RefactoringMiner, PyDriller) is operational, and the calibration target (`refactoring-toy-example`) has been mathematically verified as a high-signal environment for testing the prioritization heuristics.

Chapter 1

The Research Problem

1.1 The “Noise” Problem in Static Analysis

Code smells—symptoms of poor design or implementation choices—are prevalent in software systems. While tools exist to detect them, they suffer from a crippling Signal-to-Noise Ratio. A typical run of a static analyzer on a mid-sized enterprise project may yield 5,000+ warnings.

For a development team with limited resources, this unranked list is non-actionable. Without a mechanism to distinguish between a theoretical violation of a style guide and a “ticking time bomb” that impedes future development, teams often choose to disable the tools entirely.

1.2 The Flaw in Existing Ground Truths

To solve the noise problem, researchers have attempted to build machine learning models to classify smells. However, these models require labelled training data (Ground Truth).

The Critique: Existing datasets (e.g., Landfill, DACOS) are fundamentally flawed because they are constructed using **Subjective Manual Validation**.

1. **Decontextualization:** Experts are asked to judge isolated code snippets without knowledge of the system’s architecture or history.
2. **Inconsistency:** Inter-rater agreement among experts regarding code smells is notoriously low (Palomba et al., 2016).
3. **Theoretical vs. Practical:** An expert might label a class as a “God Class” because it violates a metric threshold, even if that class works perfectly and never requires maintenance.

1.3 The Core Argument: Revealed Preference

This research proposes a paradigm shift in defining “severity.” A code smell is only high-priority if it negatively impacts the software lifecycle.

We apply the economic theory of **Revealed Preference**:

“The true preferences of an agent are revealed by their actions, not their stated opinions.”

In software engineering terms: **The true priority of a code smell is revealed by the objective behavior of the development team.**

- If a team ignores a “God Class” for 10 years, it is—by definition—Low Priority (Benign).
- If a team halts feature development to refactor a “Long Method,” it is—by definition—High Priority.
- If a “Feature Envy” smell is the root cause of a critical bug, it is—by definition—High Priority.

Chapter 2

Research Questions & Objectives

The research is guided by one foundational question for Phase 1:

RQ1: *How can a legitimate, large-scale ground truth for code smell prioritization be automatically generated from software repository history using a fortified, multi-layered set of heuristics?*

Objectives:

1. **Design** a hybrid analysis pipeline that combines Static Analysis (Structure) and Mining Software Repositories (History).
2. **Implement** “Fortified Heuristics” that score smells based on Refactoring History, Bug Causality, and System Criticality.
3. **Validate** the pipeline using a “Calibration Target” (Toy Project) to ensure heuristic accuracy before scaling to a “Oracle Project.”

Chapter 3

Methodology (The Hybrid Orchestrator)

To answer RQ1, we are building “Smell-Ranker,” a Python-based pipeline that orchestrates external analysis tools. The methodology follows a “**Pipeline-as-Metric-Extractor**” approach, divided into three passes.

3.1 Overview: The Pipeline-as-Metric-Extractor

The system does not rely on a single tool. Instead, it acts as a synthesizer that joins data from disparate sources (AST Snapshots + Git Logs).

3.2 Phase 1: Candidate Generation (Static Analysis)

- **Goal:** Establish the “Potential Search Space.”
- **Tool:** **PMD** (Source Code Analyzer).
- **Logic:** The system checks out specific commits (e.g., HEAD or pre-refactoring states) and runs PMD to identify all theoretical smells (God Class, Long Method, Feature Envy).
- **Output:** An XML report listing every location where a metric threshold is violated.

3.3 Phase 2: Evidence Gathering (Historical Analysis)

- **Goal:** Gather objective evidence of developer behavior.
- **Tools:**
 - **RefactoringMiner:** Detects atomic refactoring operations (e.g., `Extract Method`, `Move Class`) with 100% precision.
 - **PyDriller:** Mines commit metadata, churn, and file ownership.
 - **NLP Classifier:** Scans commit messages for keywords (`fix`, `bug`, `resolve`) to identify defect-correcting activities.
- **Output:** A JSON/CSV stream of historical events linked to specific file paths.

3.4 Phase 3: The Fortified Heuristics (Synthesis)

This is the core novelty of the thesis. The system merges Phase 1 and Phase 2 data using custom logic:

3.4.1 Heuristic A: The Weighted Refactoring System

- *Flawed Approach:* Treating all refactorings equally.
- *Fortified Approach:* Complex refactorings that require significant cognitive load (e.g., `Extract Class`, `Inheritance to Delegation`) are weighted higher than simple atomic changes (`Rename Method`).
- *Justification:* Tufano et al. (2020) demonstrated that different refactoring types correlate with different levels of technical debt repayment.

3.4.2 Heuristic B: AST-Based Proximity Analysis

- *Flawed Approach:* Linking a bug to a smell if they exist in the same file.
- *Fortified Approach:* The bug-fix must overlap with the **exact line range (AST Node)** of the detected smell.
- *Justification:* This ensures causality. We define a smell as “Fix-Inducing” only if the fix modified the smelly code itself.

3.4.3 Heuristic C: Code Criticality (The Survivor Check)

- *Flawed Approach:* Assuming old smells are benign.
- *Fortified Approach:* A “Survivor” smell (one that is never fixed) is checked against the **Dependency Graph**. If the smelly class is a central hub (High Fan-In/Fan-Out), it is marked as “Dormant-High-Risk.”
- *Justification:* Zimmermann et al. showed that graph centrality is a strong predictor of defect propagation.

Chapter 4

System Architecture (“Smell-Ranker”)

The engineering backbone of this research is a modular, cloud-native pipeline designed for reproducibility.

4.1 The Hybrid Cloud Environment

- **Runtime: Google Colab** (Pro). Chosen for its disposable compute power and pre-installed data science libraries.
- **Persistence: Google Drive**. The “Hard Drive” of the system. All repositories, tools, and output datasets are stored here to survive Colab session timeouts.
- **Version Control: GitHub**. The source code of the pipeline is versioned remotely and synced to Colab at runtime.

4.2 Software Design Patterns

The codebase adheres to strict software engineering principles:

- **Facade Pattern:** `main.py` and `exec_pipeline.sh` act as simple entry points, hiding the complexity of the subsystems.
- **Adapter Pattern:** `cmd_runner.py` wraps the command-line interfaces of PMD and Refactoring-Miner, providing a clean Python API for execution and error handling.
- **Separation of Concerns:**
 - `bin/` handles Environment (Shell).
 - `pipeline/` handles Logic (Python).
 - `config.py` handles Configuration.

4.3 Toolchain Selection: The JDeodorant vs. PMD Decision

Initially, JDeodorant was considered for static analysis. However, we formally pivoted to **PMD** for the following reasons:

1. **Feasibility:** JDeodorant is an Eclipse Plugin, making headless automation in a Linux CI/CD environment (like Colab) extremely brittle. PMD provides a native CLI.
2. **Maintenance:** PMD is actively maintained (v7.18.0), whereas JDeodorant has not seen significant updates, posing risks for modern Java syntax parsing.

Chapter 5

Experimental Design & Data Selection

To ensure the scientific validity of the generated ground truth, we employ a **Multiple-Case Study** design with a strict **Calibration-First** strategy.

5.1 The “Calibration-First” Strategy

We do not begin by mining complex projects. We utilize a **Toy Project** to calibrate the heuristics.

- **Role of Toy Project:** It serves as a “Clean Room.” Because its history is known and verifiable, we can mathematically prove that the pipeline is detecting refactorings correctly before introducing noise.
- **Role of Oracle Project:** (Future Phase) A real-world project (e.g., *GildedRose* or *Commons-Lang*) will serve as the actual source of training data once the pipeline is calibrated.

5.2 The Selected Calibration Target: `refactoring-toy-example`

We selected <https://github.com/danilofes/refactoring-toy-example>.

- **Criterion 1 (Atomic Refactoring):** PASS. The commit history consists of “Root Canal Refactorings” (Pure), not “Floss Refactorings” (Mixed with logic).
- **Criterion 2 (High Signal):** PASS. The project is dense with refactoring events.

5.3 Phase 0 Mining Results (The Verification)

On **December 12, 2025**, we executed the `repo_metrics.py` mining script (powered by PyDriller) to scientifically verify the quality of the calibration target.

The Results:

```
Project: toy_project
[History] Total Commits:      65
[History] Age (Days):        243 days

[Content] Main Language:     .java
[Content] Java Files:        20
[Content] Total Churn:        1304 lines (Avg 20.1/commit)

[Heuristic] Bug-Fixes:        0 (0.0%)
[Heuristic] Refactorings:     40 (61.5%)
```

[Heuristic] Bus Factor: 1.10 authors/file
[Heuristic] Top Coupling: Feline.java + Tiger.java (8 shared commits)

Interpretation:

1. **Refactoring Density (61.5%):** This confirms the repository is “Hyper-Saturated” with training examples. It is perfect for testing Heuristic A (Weighted Refactoring).
2. **Bug-Fixes (0%):** This confirms the repository is “Clean.” There is no bug-fix noise to confuse the initial calibration.
3. **Coupling (Feline+Tiger):** This proves the existence of “Design Smells” (Feature Envy), validating that the code has sufficient structural complexity for testing.

5.4 The “Forced-Loop” Validation & Scope Refinement

During Phase 1 execution, a divergence was observed between the total repository history (65 commits) and the analysis scope of the RefactoringMiner adapter (40 commits). This section details the architectural decisions resolving this discrepancy.

5.4.1 The “Blind Spot” Problem

The default behavior of RefactoringMiner (invoked via the `-a` flag) is to analyze the linear history reachable from the tip of the `main` branch. This approach poses two risks for a ground-truth generator:

1. **Branch blindness:** It excludes detached heads, orphan branches, and abandoned feature branches that may contain valid refactoring data.
2. **Noise accumulation:** It processes documentation updates, build script changes, and metadata edits that contain no source code changes.

5.4.2 Resolution: The Forced-Loop Strategy

To control the analysis scope precisely, we implemented a custom orchestration logic in the `refm_adapt.py` adapter. Instead of relying on the tool’s internal traversal, the pipeline performs an external “Forced-Loop”:

1. **Universal Object Retrieval:** The script executes `git rev-list --all --reverse` to retrieve the SHA-1 hash of every commit object in the database, regardless of branch topology.
2. **Java-Only Filtering:** We append the `-- *.java` pathspec to the git command.
 - *Justification:* For the `toy_project`, our objective is to measure code evolution. Commits that only modify `README.md` or `.gitignore` introduce “process noise” without contributing to the “refactoring signal.”
3. **Explicit Invocation:** The adapter iterates through the filtered list (40 commits), invoking RefactoringMiner explicitly on each hash using the `-c` flag.

Outcome: This strategy successfully identified that while the project history is 65 commits long, only **40 commits (61.5%)** contain Java source code activity. The pipeline analyzed 100% of these relevant commits.

5.5 Pipeline Execution Workflow

The “Smell-Ranker” pipeline operates in three distinct layers to ensure reproducibility and data integrity:

1. **Layer 1: Infrastructure (Shell):** The `exec_pipeline.sh` script initializes the runtime. It mounts Google Drive, installs the Java 17 runtime, and enforces `chmod +x` permissions on the analysis binaries.
2. **Layer 2: Synchronization (Git):** The environment is synced with the latest `main` branch from GitHub, ensuring that the local execution environment mirrors the remote repository state.
3. **Layer 3: Orchestration (Python):** The `main.py` facade triggers the analysis modules:
 - **Phase 0 (Mining):** `repo_mets.py` calculates the global denominator (Total History = 65).
 - **Phase 1 (Analysis):** `refm_adapt.py` generates the raw `refactorings_toy_project.json`.
 - **Phase 1 (Metrics):** `refm_mets.py` aggregates the raw data into high-level indicators.

5.6 Metrics Definitions & Justification

To validate the quality of the ground truth, we derive three key metrics in `refm_mets.py`. These metrics determine if a repository is suitable for training prioritization models.

5.6.1 1. Refactoring Density (ρ)

Measures the “saturation” of the dataset. A low density implies the project is stagnant or feature-heavy, making it a poor candidate for studying design evolution.

$$\rho = \left(\frac{C_{ref}}{C_{total}} \right) \times 100$$

Where C_{ref} is the count of commits containing at least one refactoring, and C_{total} is the total project history (65).

5.6.2 2. Signal Strength (σ)

Measures the intensity of refactoring activity per relevant commit.

$$\sigma = \frac{\sum Ops}{C_{ref}}$$

5.6.3 3. Commit Purity (P)

This metric distinguishes between “Root Canal Refactoring” (Pure) and “Floss Refactoring” (Mixed with feature work). We employ a heuristic based on Code Churn. A commit is flagged as “Impure” if the total lines modified (L_{churn}) exceed a threshold relative to the refactoring operations (O_{count}):

$$\text{Impure} \iff L_{churn} > (O_{count} \times 20)$$

$$P = \left(\frac{C_{ref} - C_{impure}}{C_{ref}} \right) \times 100$$

Justification: High impurity introduces noise. If a bug is introduced in an impure commit, it is difficult to isolate whether the refactoring or the feature addition caused it.

5.7 Phase 1 Results: The Calibration Metrics

The pipeline successfully generated `refactoring_metrics_toy_project.json`. The results for the calibration target are as follows:

- **Refactoring Density: 61.5% (40/65).**
 - *Verdict:* Excellent. The dataset provides high signal coverage.
- **Commit Purity: 52.5%.**
 - *Verdict:* Moderate. Nearly half of the refactorings are “Floss Refactorings,” accompanied by significant code churn. This is expected in a toy project used for teaching, where examples are rewritten entirely between commits.
- **Dominant Refactoring Types:**
 - Pull Up Method (33 occurrences)
 - Pull Up Attribute (13 occurrences)
 - Push Down Method (12 occurrences)
- *Interpretation:* The data clearly reflects an evolution of inheritance hierarchies (e.g., extracting `Animal` superclass), confirming the project is suitable for testing “Design Smell” detectors.

Chapter 6

Current Project Status

As of December 16, 2025:

- **Phase 0 (Infrastructure): COMPLETED.**
 - Colab/Drive/GitHub integration is fully operational.
 - Shell orchestration scripts (`exec_pipeline.sh`) are stable.
 - Dependency injection (Java 17, PyDriller) is automated.
- **Phase 1 (Implementation): COMPLETED / VERIFIED.**
 - **Adapter Architecture:** Implemented `refm_adapt.py` with dynamic output naming to prevent data collisions.
 - **Scope Refinement:** Implemented “Forced-Loop” with Java filtering, successfully bridging the history gap.
 - **Metric Logic:** `refm_mets.py` and `repo_mets.py` now export standardized JSON metrics.
 - **Smoke Test:** Successfully executed the full pipeline on `toy_project`.
- **Phase 2 (PMD Integration): IN PROGRESS.**
 - The `main.py` facade is configured to import `pmd_adapt` dynamically.
 - Currently waiting for the Mentee (Darun) to merge the PMD logic into the `dev` branch.

Appendix A: Daily Research Log

Entry: Dec 11, 2025

- **Task:** Completed Infrastructure Documentation.
- **Decision:** Finalized folder structure for Google Drive persistence (`Thesis_Project/repos/`).
- **Outcome:** Created `Smell Ranker_Infrastructure And Architecture Documentation.md`.

Entry: Dec 12, 2025

- **Task:** Phase 0 Data Mining (Repository Verification).
- **Action:** Updated `repo_metrics.py` to use the `PyDriller` library instead of raw Git commands.
- **Action:** Modified `colab_env_setup.sh` to auto-install `pydriller` during runtime initialization.
- **Result:** Mined `toy_project`. Confirmed 61.5% refactoring density. This scientifically justifies the selection of this repo for calibration.

Entry: Dec 13, 2025

- **Task:** Phase 1 Architecture & Team Workflow Setup.
- **Action:** Refactored project structure to use the Adapter Pattern (`pipeline/adapters/`) for better modularity.
- **Action:** Implemented `main.py` as the central facade to orchestrate `refm_adapt.py` and `pmd_adapt.py`, including robust error handling.
- **Action:** Defined and documented a Git Feature Branch Workflow to assign the PMD integration task to Mentee (Darun).
- **Result:** Core "Smoke Test" architecture is committed to the `dev` branch.

Entry: Dec 15, 2025

- **Task:** Phase 1 Architecture & Team Workflow Setup.
- **Action:** Resolved the "Commit Gap" issue (38 vs 65) by implementing a "Forced-Loop" strategy in `refm_adapt.py`, achieving 100% history coverage.
- **Result:** Core "Smoke Test" architecture is committed to the `dev` branch.

Entry: Dec 16, 2025

- **Task:** Final Phase 1 Verification.
- **Action:** Updated `refm_adapt.py` to filter for `*.java` files, refining the analysis scope to 40 relevant commits.
- **Action:** Implemented `refm_mets.py` to calculate Density, Signal Strength, and Purity.
- **Result:** Pipeline executed successfully. Generated `refactoring_metrics_toy_project.json`. Confirmed 61.5% density and valid JSON output structure. Ready for PMD integration.